로그인 폼 진행 내용

- 날짜: 2026-11-02
- 대상: Next.js 16.4.3, React 19.3.0, `lgcns-fe-next`
- 기준은 현재 소스다. `/login`은 서버 함수와 `FormData`로 제출하고, 화면 값과 오류는 클라이언트 상태로 둔다. `/client-login`은 제출을 브라우저에서 끝낸다.

`handleSubmit` 안의 `// api call`은 주석이다. `fetch`나 `XMLHttpRequest`는 두 로그인 파일에 없다.

1. 파일과 선언

파일	선언	모듈	하는 일
app/(auth)/login/page.tsx	async function handleSubmit + "use server"	서버 컴포넌트	FormData에서 아이디와 비밀번호를 읽어 서버 로그에 찍는다
component/templates/LoginSection.tsx	function LoginSection	서버 컴포넌트	서버 함수를 유기체로 넘기는 <section>
component/organisms/LoginForm.tsx	"use client" + function LoginForm	클라이언트 컴포넌트	useState로 값과 오류를 두고, next/form의 action에 서버 함수를 넣는다
component/molecules/FormField.tsx	'use client' + function FormField	클라이언트 컴포넌트	입력, 지우기 버튼, 비밀번호 보기 버튼을 한 필드로 묶는다
component/atoms/ErrorMessage.tsx	function ErrorMessage	클라이언트 그래프	지시문은 없다. FormField가 가져온다. 오류가 없어도 <p>를 그린다
schema/loginSchema.ts	function loginFieldMessage	클라이언트 그래프	LoginForm만 가져온다. 브라우저에서 Zod로 필드 메시지를 만든다
types/request/RequestType.ts	interface UserPayloadType	타입	userId, password가 string이다
types/initial/loginInitial.ts	const initialValues	값	두 필드의 처음 값은 ""이다. 기본 내보내기다
types/error/loginError.ts	export const emptyErrors	값	두 필드의 처음 오류는 ""이다. 이름 있는 내보내기다
app/(auth)/client-login/page.tsx	'use client' + function ClientLogin	클라이언트 컴포넌트	useState만 읽고, preventDefault 뒤 브라우저 로그에 찍는다
app/(auth)/layout.tsx	function AuthLayout	서버 컴포넌트	두 페이지를 감싼다

파일	선언	모듈	하는 일
components/ui/input.tsx	function Input	클라이언트 경계	export는 이름 있는 내보내기다. @base-ui/react/input의 Input.js 첫 줄이 'use client'다
components/ui/button.tsx	function Button	클라이언트 경계	export { Button, buttonVariants }. 기본 내보내기는 없다
next/form	import Form from "next/form"	클라이언트 모듈	next/dist/client/form.js 첫 줄이 'use client'다

(auth)는 라우트 그룹이다. URL에는 없다. 주소는 /login과 /client-login이다.

handleSubmit의 async는 ECMAScript async 함수다. 호출 결과가 프로미스다. HTML의 <script async>와 같은 속성이 아니다. Next.js 문서에서 서버 함수는 비동기여야 한다. 서버에서 실행되고, 클라이언트가 네트워크 요청으로 부르기 때문이다.

"use server"는 그 함수를 서버 함수로 표시하는 지시문이다. 이 페이지에서는 함수 본문 첫 줄에 있다. "use client"와 'use client'는 그 파일이 클라이언트 컴포넌트 그래프의 시작이라는 지시문이다. 따옴표 종류는 동작 차이가 아니다. 두 지시문은 호이스팅 문법이 아니다.

components/ui/button.tsx는 Button을 이름 있는 내보내기로만 제공한다. LoginForm은 import { Button } from "@/components/ui/button"으로 가져온다. 기본 내보내기로 가져오면 그 모듈에 default가 없다는 오류가 난다.

2. 로딩 시점

이 로그인 파일은 HTML에 <script src>로 적혀 있지 않다. Next가 서버에서 서버 컴포넌트를 실행하고, "use client"로 시작한 그래프만 브라우저 모듈로 보낸다.

코드	적힌 곳	로딩
`<script src="js/main.js">`	`lgcns-fe-html/index.html`, `about.html` 본문 끝	속성 없음. 파서가 그 지점에서 스크립트를 받아 실행한 뒤 다음 HTML로 간다. 동기 실행이다
`<script src="js/main.js" async>`	`lgcns-fe-html/contact.html`	`async`. 받는 동안 파서는 계속 가고, 받으면 바로 실행한다. 여러 스크립트의 실행 순서는 문서 순서가 아니다
`<script src defer>`	이 저장소의 로그인 HTML에는 없다	`defer`는 받는 동안 파서를 막지 않고, 문서가 파싱된 뒤 문서 순서대로 실행한다
`<script type="module" src="/src/main.tsx">`	`lgcns-fe/index.html`	`type="module"`. 모듈 스크립트는 지연 실행된다. `defer`를 적어도 모듈의 기본 동작과 같다
`async function handleSubmit`	`app/(auth)/login/page.tsx`	브라우저 스크립트 태그가 아니다. `"use server"` 때문에 제출 때 서버에서 실행된다
`'use client'` `FormField`, `ClientLogin`	클라이언트 컴포넌트	Next가 클라이언트 모듈로 보낸다. `onClick`과 `onSubmit`은 브라우저에서 실행된다

서버 컴포넌트인 `login/page.tsx`와 `LoginSection.tsx`의 함수 본문은 브라우저 번들로 내려가지 않는다. 그 안의 서버 함수만 클라이언트가 부를 수 있는 참조로 넘어간다. `LoginForm`이 `FormField`, `loginSchema`, `loginInitial`, `loginError`를 import 하므로 그 파일들도 클라이언트 모듈에 실린다. `ErrorMessage.tsx`에는 지시문이 없어도 `FormField`가 가져오면 같은 그래프에 들어간다.

3. 아토믹 계층

계층	파일	아는 것
원자	`ErrorMessage`, `components/ui/input`, `components/ui/button`	문단 하나, 입력 하나, 버튼 하나
분자	`FormField`	입력과 그 필드의 버튼, 오류 문단
유기체	`LoginForm`	두 필드의 상태, Zod, 제출 버튼, 서버 함수 연결
템플릿	`LoginSection`	유기체를 `<section>`에 둔다
페이지	`app/(auth)/login/page.tsx`	서버 함수를 만들어 템플릿에 넘긴다

페이지의 헤더와 푸터 주석은 주석이다. 헤더와 푸터 컴포넌트는 이 파일에 없다.

4. /login: 서버 함수와 FormData

```
async function handleSubmit(formData: FormData) {
  "use server";
  const userId = formData.get("userId");
  const password = formData.get("password");
  console.log("userId: ", userId, "password: ", password);
}
```

LoginSection이 이 함수를 LoginForm에 넘긴다. LoginForm은 <Form action={handleSubmit}>을 그린다. Next.js 문서는 서버 함수를 클라이언트 컴포넌트 안에 정의할 수 없고, 서버 컴포넌트에서 만들어 props로 넘길 수 있다고 적는다. "use server"가 없는 함수를 이 action에 넣으면, 클라이언트 컴포넌트인 Input, Button, Form 쪽으로 함수가 넘어가 프리렌더가 실패한다.

1. 입력이 바뀔 때마다 LoginForm의 handleChange가 브라우저에서 실행된다. name이 userId나 password가 아니면 반환한다. 아니면 payload와 errors를 갱신하고, name과 value를 브라우저 콘솔에 찍는다.
2. loginFieldMessage는 그 필드만 loginSchema.shape[field].safeParse 한다. 빈 문자열이면 "아이디를 입력해 주세요." 또는 "비밀번호를 입력해 주세요."를 반환한다. 성공하면 ""를 반환한다.
3. loginSchema.safeParse(payload).success가 거짓이면 제출 버튼 disabled가 참이다. 스키마는 각 문자열이 한 글자 이상이어야 성공한다.
4. 버튼이 눌리면 Form의 action이 함수이므로 Next.js Form은 React 폼처럼 그 서버 함수를 실행한다.
5. 서버 함수의 인자는 React 상태가 아니다. HTML 폼이 이름이 있는 성공한 컨트롤로 만드는 FormData다. FormField의 입력에 name="userId", name="password"가 있다.
6. formData.get은 그 키의 첫 값을 반환한다. 이 소스는 반환 타입이 문자열인지 검사하지 않고 console.log만 한다. 로그는 서버 프로세스로 나간다. 개발 서버에서는 POST /login 다음에 두 값이 찍힌다.

입력은 통제 컴포넌트다. value={payload.userId}처럼 React 상태가 화면에 보이는 값을 정한다. 같은 입력이 name도 가지므로 제출 시점의 DOM 값이 FormData에 들어간다. 서버 함수는 payload를 받지 않고 loginSchema도 호출하지 않는다. Zod는 버튼과 오류 문구를 위한 클라이언트 동작이다.

handleSubmit은 인증, 쿠키, redirect, revalidatePath를 하지 않는다.

5. 필드 버튼

FormField는 type === "password"일 때만 보기 버튼을 그린다. useState(false)인 isVisible이 참이면 입력 type은 "text"이고, 거짓이면 받은 type 그대로다. 버튼의 aria-pressed는 isVisible이다. 문구는 보일 때 "비밀번호 숨기기", 가려져 있을 때 "비밀번호 보기"다. 아이콘은 lucide-react의 Eye와 EyeOff다.

isClearable이 참이고 value가 빈 문자열이 아니면 X 버튼을 그린다. /login의 아이디와 비밀번호 필드는 둘 다 isClearable={true}다. 클릭하면 onChange에 value: ""인 객체를 넘긴다. LoginForm의 handleChange가 그 name과 value만 읽으므로 상태가 비워지고, Zod가 빈 값 메시지를 errors에 넣는다. 이 객체는 브라우저가 만든 ChangeEvent가 아니다. target만 흉내 낸 값이다.

두 버튼 모두 type="button"이다. 폼 안의 <button>은 type을 생략하면 제출 버튼이 된다. type="button"이면 폼을 보내지 않는다.

비밀번호 필드에는 보기 버튼이 오른쪽 `right-1.5`, 지우기 버튼이 `right-7`에 있다. 입력 `className`은 버튼이 둘이면 `"pr-14"`, 하나면 `"pr-8"`이다. 아이디 필드는 지우기 버튼만 있으므로 값이 있을 때 `"pr-8"`이다.

6. 오류 문단

`ErrorMessage`는 `error`가 없어도 `null`을 반환하지 않는다. 그 분기는 주석이다. 항상 `<p role="alert">`를 그린다. `style`의 `opacity`는 `error`가 있으면 `1`, 없으면 `0`이다. `transition`은 `"opacity 0.3s ease-in-out"`이다. 글자가 없어도 `py-4`인 문단은 DOM에 남는다. `id`는 `FormField`가 만든 `${id}-error`다. 입력이 오류 문자열을 가지면 `aria-invalid`가 참이고 `aria-describedby`가 그 `id`를 가리킨다.

7. /client-login: 브라우저에서 끝내기

파일 첫 줄이 `'use client'`다. 서버 함수는 없다.

```
const handleSubmit = (e: React.FormEvent<HTMLFormElement>) => {
  e.preventDefault();
  console.log("userId: ", userId, "password: ", password);
};
```

키 입력마다 `setUserId` 또는 `setPassword`가 상태를 바꾸고, 그 전에 `e.target.value`를 브라우저 콘솔에 찍는다. 입력에 `name`이 없다. 제출하면 `preventDefault`가 폼의 기본 전송을 막는다. `FormData`를 만들지 않고 상태 값을 로그에 남긴다. 이 제출로 `POST /client-login`이 생기지는 않는다. Zod, 지우기 버튼, 보기 버튼은 이 파일에 없다.

8. 두 화면 비교

	/login	**/client-login**
페이지 선언	서버 컴포넌트. 안에 `"use server"` 함수	파일 전체 `'use client'`
폼	`next/form`의 `<Form action={handleSubmit}>`	HTML `<form onSubmit={handleSubmit}>`
값이 사는 곳	제출 값은 `FormData`. 화면 값과 오류는 `useState`	화면 값과 제출 값이 같은 `useState`
입력 이름	`name="userId"`, `name="password"`	`name` 없음
제출 시	서버 함수가 `POST`로 실행되고 서버 로그가 남는다	`preventDefault` 후 브라우저 로그만 남는다
검사	클라이언트 Zod. 실패하면 제출 버튼 `disabled`	검사 코드 없음
필드 버튼	값이 있으면 X. 비밀번호는 보기 전환	없음

`/login`도 클라이언트 상태를 쓴다. 차이는 제출의 목적지가 서버 함수인지, 브라우저 이벤트에서 끝나는지다.

9. 명세와 공식 문서

HTML 폼 제출은 이름이 있는 성공한 컨트롤로 폼 데이터 셋을 만든다. `name`이 없는 컨트롤은 그 셋에 들어가지 않는다. `FormData.get(name)`은 그 키의 첫 값을 반환하고, 없으면 `null`이다. 텍스트 입력이면 값은 문자열이다.

`preventDefault`는 이벤트의 기본 동작을 취소한다. 폼 `submit`의 기본 동작은 폼을 보내는 것이다.

`<button>`의 `type`을 생략하면 폼 안에서 제출 버튼이다. `type="button"`은 기본 동작이 없다. 비밀번호 입력의 `type="password"`는 값을 가리고, `type="text"`는 그대로 보여 준다. `aria-pressed`는 토글 버튼이 눌린 상태인지 나타낸다.

React 19의 `<form action>`은 함수를 받으면 그 함수에 `FormData`를 넘긴다. `useState`는 그 컴포넌트가 다시 렌더될 때 같은 상태 값을 돌려준다. 훅은 클라이언트 컴포넌트 함수의 최상위에서 호출한다.

Next.js는 폼 `action`에 넘긴 서버 함수를 `POST`로만 호출한다. 서버 컴포넌트 안에 두려면 함수 본문 첫 줄에 `'use server'`를 둔다. `import Form from "next/form"`은 `action`이 문자열이면 GET 검색 이동이고, 함수이면 그 서버 함수를 실행한다. 이 프로젝트의 `LoginForm`은 함수를 넘긴다.

Zod `z.string().min(1, message)`는 길이가 1보다 작으면 그 메시지를 이슈로 둔다. `safeParse`가 실패하면 `error.issues`에 이슈가 있다. 이 스키마의 `min` 첫 인자는 숫자 `1`이고, 두 번째 인자는 문자열 메시지다.

스크립트 로딩 속성은 HTML script 요소다. `async`와 `defer`는 외부 스크립트의 가져오기와 실행 시점을 정한다. `type="module"` 스크립트는 지연된다. Next 앱 라우터의 서버 컴포넌트와 서버 함수는 그 태그를 페이지에 직접 적지 않는다.

서버가 만든 HTML과 브라우저가 처음 그리는 트리가 다르면 React는 그 트리를 클라이언트에서 다시 만든다. 입력을 감싼 `<div class="relative">`가 서버 HTML에만 있고 브라우저 번들에는 아직 없으면, `FieldControl`이 맞춰야 할 노드가 `input`이 아니라 그 `div`가 된다. 현재 `FormField`는 서버와 클라이언트 모두 그 `div`로 입력을 감싼다.

- https://developer.mozilla.org/en-US/docs/Web/API/FormData
- https://xhr.spec.whatwg.org/#interface-formdata
- https://developer.mozilla.org/en-US/docs/Web/API/Event/preventDefault
- https://developer.mozilla.org/en-US/docs/Web/HTML/Element/button
- https://developer.mozilla.org/en-US/docs/Web/HTML/Element/input
- https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/Reference/Attributes/aria-pressed
- https://react.dev/reference/react-dom/components/form
- https://react.dev/reference/react/useState
- https://nextjs.org/docs/app/getting-started/mutating-data
- https://nextjs.org/docs/app/api-reference/components/form
- https://zod.dev
- https://html.spec.whatwg.org/multipage/scripting.html#the-script-element
- https://nextjs.org/docs/messages/react-hydration-error

10. 이 소스에 없는 것

- `/login`과 `/client-login`의 `fetch`, `XMLHttpRequest`, 실제 API 호출. `// api call`은 주석이다
- 서버 함수 안의 Zod 검사, 세션 확인, 쿠키 저장, `redirect`, `revalidatePath`
- `useActionState`, `useFormStatus`
- `LoginHeader`, `LoginFooter` 컴포넌트